
Introduction to Python Programming

HUDM 5001 Section 001 Week 01

Chenguang Pan

Sept. 08, 2026



Today's Plan

1. Who I am, and who you are
2. What Python is, and what it can do
3. Why we still learn Python in the age of AI
4. How we'll learn it in this course
5. The Syllabus: schedule, grading, and AI policy
6. Understanding Colab: Markdown and your first line of Python

1.0 Who I am, and who you are

1.1 About me

Chenguang Pan

PhD candidate in *Measurement, Evaluation, & Stats*

My fifth year at TC, advised by Dr. Youmi Suk

Nine years in the Edu & Tech industry

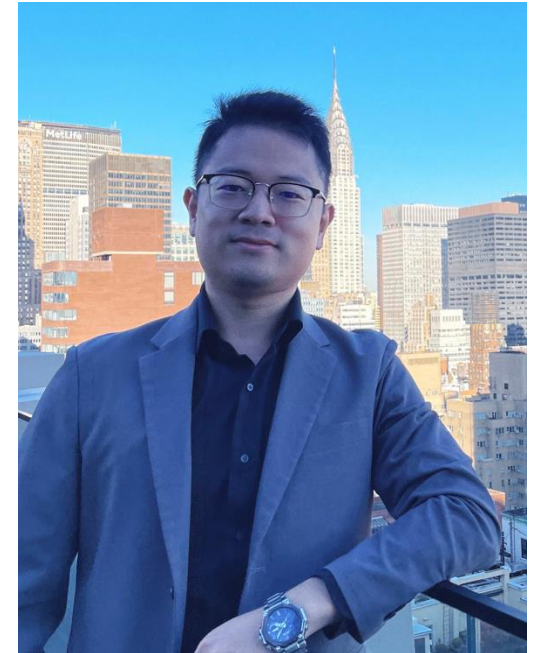
Causal ML, GenAI for synthetic data, Agentic System

Email: cp3280@tc.columbia.edu

Personal website: cpan.ai

Lab: youmilab.ai

An auto-research system: edmars.ai



1.2 About you

Who has used Excel or Google Sheets for data?

Then you already think in rows, columns, and variables.

Who has used SPSS, Stata, SAS, or R?

Then you already think about analysis as a procedure.

Who has written any Python at all?

You'll get depth from the pandas, SQL, Shell, and GitHub units.

Who is a little nervous about coding?

This course assumes zero programming background, and nervous is normal.

2.0 What Python is, and what it can do

2.1 Python in brief

A general-purpose programming language

Created by Guido van Rossum, first released in 1991

Named after a British comedy *Monty Python's Flying Circus*, not the snake.

Free and open source

Intuitive language as powerful as major competitors.



*Figure was generated by Gemini.

2.1 Python in brief



PHP. Rasmus Lerdorf



Java. James Gosling



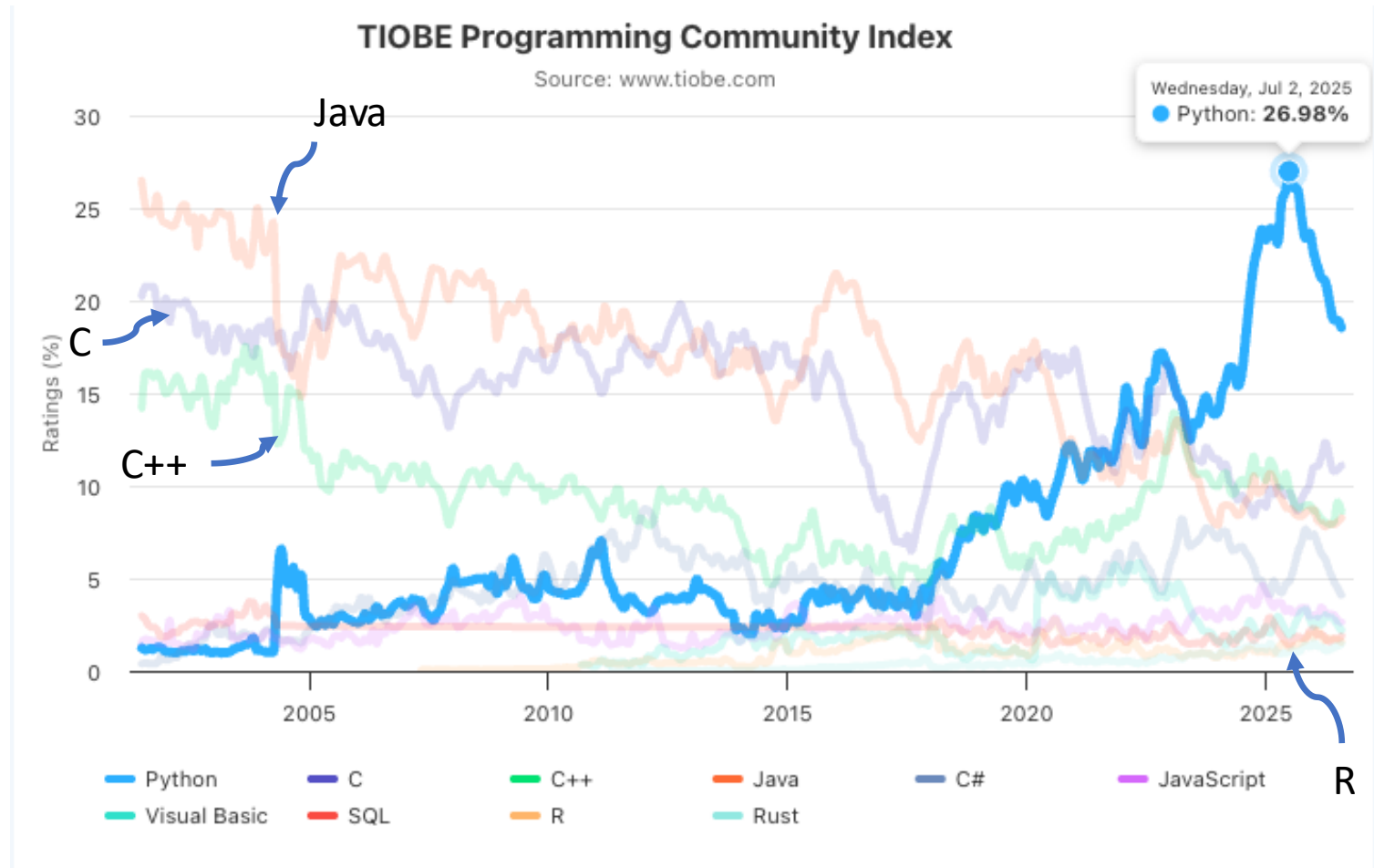
C++. Bjarne Stroustrup



Python. Guido van Rossum

2.1 Popularity: Ranking

#1



Data source: <https://www.tiobe.com/tiobe-index/>

2.1 Popularity

#1~2

most-used language on GitHub

#1

most-used language in all new AI projects. ~50% .

46%

job postings in the U.S. requiring Python skills, followed by SQL at 45%

Data source:

(1) Octoverse: [click here](#).

(2) Finopotamus: [click here](#).

2.2 Readability

Suppose we have a list of students' scores, named `scores`.

```
avg = sum(scores) / len(scores)
```

```
double sum = 0;
for (int i = 0; i < scores.length; i++) {
    sum += scores[i];
}
double avg = sum / scores.length;
```

2.2 What Python can do?

(1) Runs major apps:



“Still watching?” and the next show it picks:
recommendation systems built largely in Python.



Discover Weekly a playlist assembled for you every Monday.



“Customers also bought,” and the speech pipeline behind Alexa.



2.2 What Python can do?

(2) Data science:

Problem & Ingestion

Frame objectives, define success metrics, extract raw data...

Data Cleaning

address missing values, drop duplications, feature engineering...

Exploratory Analysis

Distributions, correlations, descriptive stats...

Model Building

Train model (ML, DL,...) on split sets, monitor the loss...

Tuning & Validation

Optimize hyperparameters, cross validation ...

Deploy & Storytell

visualizations, dashboards, ...

2.2 What Python can do?

(2) Data science:

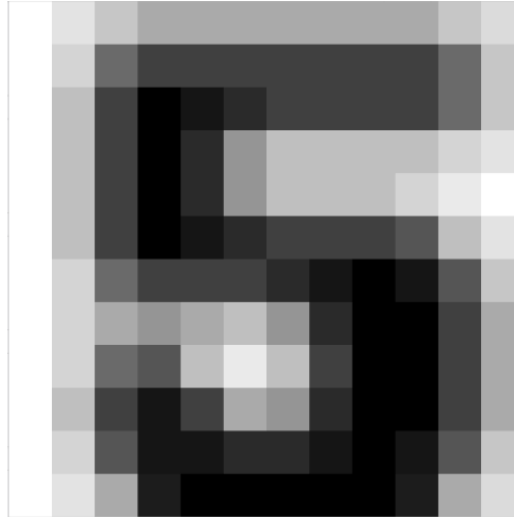
Over 90% of the computational work can be done in Python.



*Figure was generated by Gemini.

2.2 What Python can do?

(3) Computer vision: help computer see the world



149	191	191	191	191	191	191	191
191	255	234	213	191	191	191	191
191	255	213	106	64	64	64	64
191	255	213	106	64	64	64	43
191	255	234	213	191	191	191	170
149	191	191	191	213	234	255	234
85	106	85	64	106	213	255	255
149	170	64	21	64	191	255	255

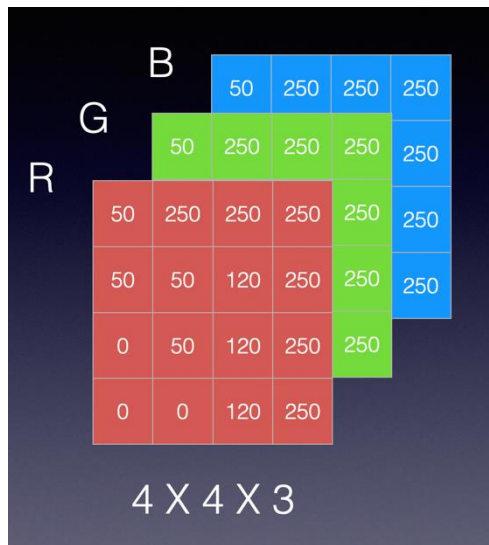
To a computer, a photo is just a grid of numbers.

Grayscale represents each pixel using 0 (black) to 255 (white)

“Seeing” means finding patterns in the grid: edges, shapes,...

2.2 What Python can do?

(3) Computer vision: help computer see the world



$$f \left(\begin{pmatrix} 93 & 92 & 83 & 77 & 77 \\ 92 & 77 & 77 & 77 & 92 \\ 92 & 77 & 83 & 77 & 92 \\ 77 & 77 & 77 & 92 & 92 \\ 77 & 77 & 92 & 92 & 92 \end{pmatrix} \right) = \begin{pmatrix} 83 & 92 & 83 & 77 & 77 \\ 99 & 99 & 77 & 77 & 92 \\ 99 & 77 & 83 & 77 & 92 \\ 77 & 77 & 77 & 95 & 92 \\ 77 & 77 & 95 & 92 & 92 \end{pmatrix} \begin{pmatrix} 93 & 92 & 83 & 69 & 69 \\ 92 & 69 & 69 & 77 & 92 \\ 92 & 69 & 83 & 77 & 92 \\ 69 & 69 & 77 & 92 & 92 \\ 77 & 77 & 92 & 92 & 92 \end{pmatrix} \begin{pmatrix} 83 & 92 & 83 & 77 & 77 \\ 83 & 77 & 77 & 77 & 92 \\ 92 & 77 & 83 & 75 & 85 \\ 75 & 77 & 75 & 85 & 85 \\ 75 & 75 & 85 & 85 & 85 \end{pmatrix}$$

*Figure source:

Left: Chenguang's 2020 slides.

Right: [freeCodeCamp](https://www.freecodecamp.org/)

2.2 What Python can do?

(3) Natural Language Processing: help computer understand human language

king - man + woman = ?

Word	Op	Dim 1	Dim 2	Dim 3	Dim 4	Dim 5
king		0.94	0.71	0.12	0.88	0.31
man	-	0.91	0.15	0.09	0.11	0.28
woman	+	0.13	0.19	0.11	0.09	0.33
	=	0.16	0.75	0.14	0.86	0.36
queen	≈	0.11	0.78	0.15	0.83	0.39

2.2 What Python can do?

It's all numbers.

Photos

grids of numbers

Words

lists of numbers

Everything else

songs, clicks, test scores, survey responses

Data science is organizing numbers and doing math on them.

3.0 Why learn Python in the age of AI

3.1 Vibe Coding



Andrej Karpathy ✓
@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

6:17 PM · Feb 2, 2025 · 7.3M Views



1.4K



6.1K



34K



17K



Relevant ▾

View quotes >

Prompt Intent



AI processing



Results

3.1 Vibe Coding

You can't check what you can't read

AI code is confidently wrong just often enough to be dangerous

Directing AI takes expertise

Good prompts and good products come from knowing what a DataFrame, a loop, and a model are

AI makes learning faster

AI is your patient, free, always-available tutor.
Use it to learn, not skip learning

Prompt Intent



AI processing



Results

3.1 Vibe Coding

“Calculators didn’t end math class. They ended math class for people who couldn’t check the calculator.”

“AI won’t replace programmers. It will replace programmers who can’t verify the AI.”

4.0 How we'll learn it in this course

4.1 Coding Environment

Notebooks-Colab, Jupyter

Code in cells, mixed with text, results, and charts.
Built for analysis, teaching, and exploration.
You run one cell at a time and see the result immediately.



IDEs- VS Code, PyCharm

An IDE (Integrated Development Environment) is the workshop where you write and run code.
Built for scripts and larger projects rather than for exploring one cell at a time.



4.1 Coding Environment



05_Control_Structures_iterables.ipynb ×

05_Control_Structures_iterables.ipynb > M HUDM 5001 — Programming for Data Science

Generate + Code + Markdown Colab Run All Clear All Outputs Outline ...

Jupyter Notebook

Select Kernel

2.2 if / elif / else — decisions

The anatomy is three parts: the keyword `if`, a **condition** (any expression that evaluates to `True/False` — exactly the comparisons from Session 02), and a **colon**, followed by an **indented block**. The indentation is not decoration: it is Python's grammar for "these lines belong to the `if`". Four spaces is the convention, and Colab indents for you when you type the colon and press Enter.

```
1 score = 72
2 attendance = 0.9
3
4 # the Session-02 pass rule, now with consequences
5 if (score >= 60) and (attendance >= 0.8):
6     print("Passed the course ✓")
7 else:
8     print("Not this time — see the make-up policy.")
```

[3] Python

... Passed the course ✓

When there are more than two possibilities, chain conditions with `elif` ("else if"). Python checks the conditions **top to bottom** and runs the block of the *first* one that is `True` — then skips the rest of the ladder entirely. An optional final `else` is the catchall for "none of the above".

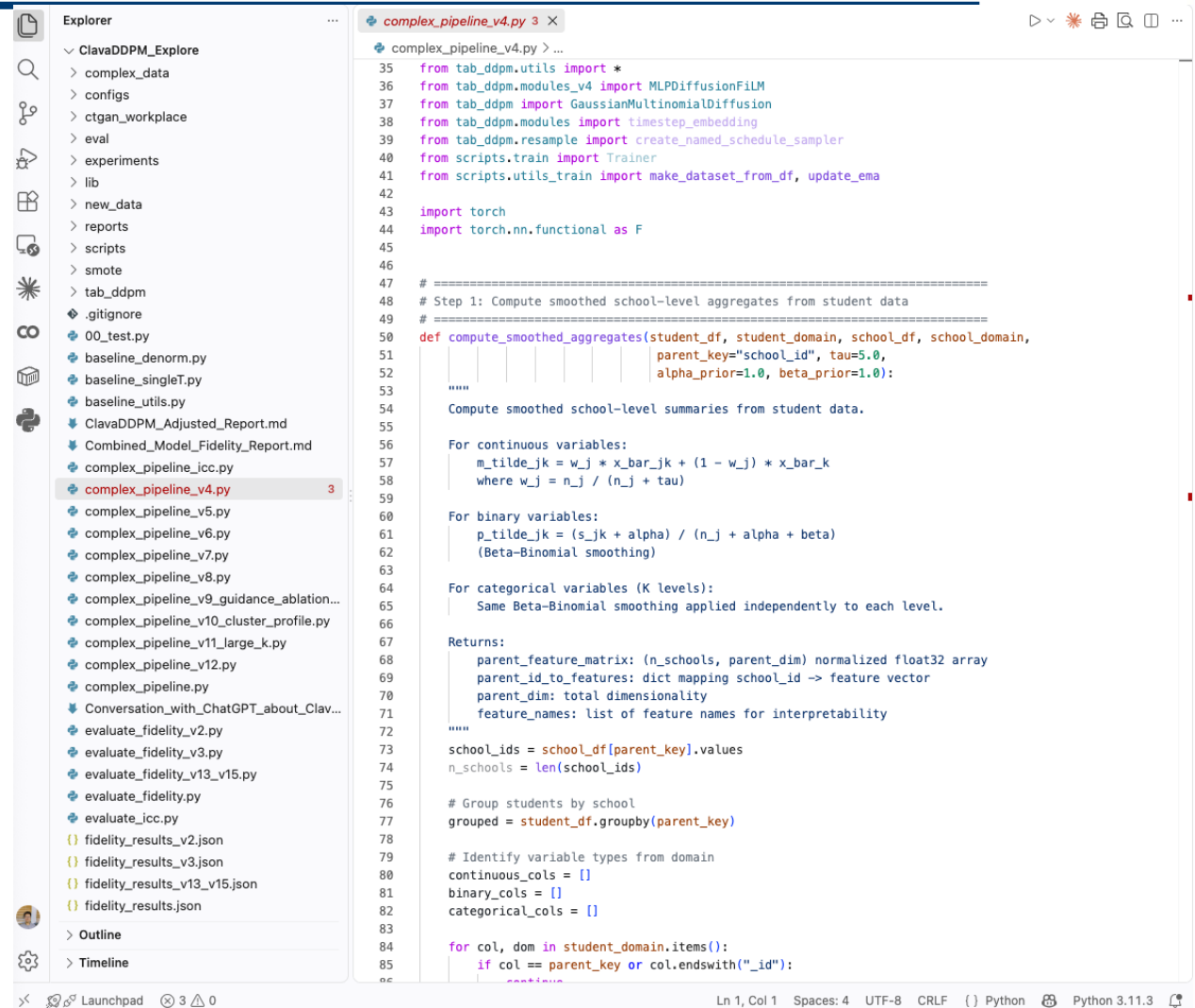
```
1 score = 87
2
3 if score >= 90:
4     grade = "A"
5 elif score >= 80:
6     grade = "B"
7 elif score >= 70:
8     grade = "C"
9 else:
10    grade = "below C"
11
12 print("score", score, "->", grade)
```

[4] Python

Launchpad 2 0

Cell 1 of 78 0.00/hr

4.1 Coding Environment



```
complex_pipeline_v4.py 3
35 from tab_ddpm.utils import *
36 from tab_ddpm.modules_v4 import MLPDiffusionFILM
37 from tab_ddpm import GaussianMultinomialDiffusion
38 from tab_ddpm.modules import timestep_embedding
39 from tab_ddpm.resample import create_named_schedule_sampler
40 from scripts.train import Trainer
41 from scripts.utils_train import make_dataset_from_df, update_ema
42
43 import torch
44 import torch.nn.functional as F
45
46
47 # =====
48 # Step 1: Compute smoothed school-level aggregates from student data
49 # =====
50 def compute_smoothed_aggregates(student_df, student_domain, school_df, school_domain,
51                                parent_key="school_id", tau=5.0,
52                                alpha_prior=1.0, beta_prior=1.0):
53     """
54     Compute smoothed school-level summaries from student data.
55
56     For continuous variables:
57         m_tilde_jk = w_j * x_bar_jk + (1 - w_j) * x_bar_k
58         where w_j = n_j / (n_j + tau)
59
60     For binary variables:
61         p_tilde_jk = (s_jk + alpha) / (n_j + alpha + beta)
62         (Beta-Binomial smoothing)
63
64     For categorical variables (K levels):
65         Same Beta-Binomial smoothing applied independently to each level.
66
67     Returns:
68         parent_feature_matrix: (n_schools, parent_dim) normalized float32 array
69         parent_id_to_features: dict mapping school_id -> feature vector
70         parent_dim: total dimensionality
71         feature_names: list of feature names for interpretability
72     """
73     school_ids = school_df[parent_key].values
74     n_schools = len(school_ids)
75
76     # Group students by school
77     grouped = student_df.groupby(parent_key)
78
79     # Identify variable types from domain
80     continuous_cols = []
81     binary_cols = []
82     categorical_cols = []
83
84     for col, dom in student_domain.items():
85         if col == parent_key or col.endswith("_id"):
```

4.2 Google Colab

[Google Colab](#) is an online version of Jupyter Notebook.

A student's account has free powerful GPU use.

4.3 Set up your local coding environment

Follow [this step-by-step instruction](#) to set up your local coding environment.

5.0 The Syllabus: schedule, grading, and AI policy

6.0 Your first line of Python: [Google Colab](#)